Program 14:- Write a program to implement water conection problem using greedy search.

```
#include <cstring>
#include <iostream>
#include <vector>
using namespace std;
int number_of_houses, number_of_pipes;
int ending_vertex_of_pipes[1100];
int diameter_between_two_pipes[1100];
int starting_vertex_of_pipes[1100];
vector<int> a, b, c;
int ans;
int dfs(int w) {
   if (starting_vertex_of_pipes[w] == 0)
      return w;
   if (diameter_between_two_pipes[w] < ans)
      ans = diameter_between_two_pipes[w];
   return dfs(starting_vertex_of_pipes[w]);
}
void solve(vector<vector<int>>& arr) {
   for (int i = 0; i < number_of_pipes; ++i) {
      int house_1 = arr[i][0], house_2 = arr[i][1], pipe_diameter = arr[i][2];
      starting_vertex_of_pipes[house_1] = house_2;
      diameter_between_two_pipes[house_1] = pipe_diameter;
      ending_vertex_of_pipes[house_2] = house_1;
   }

   a.clear();
   b.clear();
   c.clear();

   for (int j = 1; j <= number_of_houses; ++j) {
      if (ending_vertex_of_pipes[j] == 0 && starting_vertex_of_pipes[j]) {
         ans = 1000000000;
         int w = dfs(j);
         a.push_back(j);
         b.push_back(w);
         c.push_back(ans);
      }
   }
   cout << a.size() << endl;
   for (int j = 0; j < a.size(); ++j)
      cout << a[j] << " " << b[j] << " " << c[j] << endl;
}
int main() {
   cout << "Enter the number of houses: ";
   cin >> number_of_houses;
   cout << "Enter the number of pipes: ";
   cin >> number_of_pipes;
   memset(ending_vertex_of_pipes, 0, sizeof(ending_vertex_of_pipes));
   memset(starting_vertex_of_pipes, 0, sizeof(starting_vertex_of_pipes));
   memset(diameter_between_two_pipes, 0, sizeof(diameter_between_two_pipes));
```

```
    vector<vector<int>> arr(number_of_pipes, vector<int>(3));
    cout << "Enter the details of the pipes (start_house end_house diameter):\n";
    for (int i = 0; i < number_of_pipes; ++i) {
        cin >> arr[i][0] >> arr[i][1] >> arr[i][2];
    }
    solve(arr);
    return 0;
}
```

Output:-

```
Enter the number of houses: 9
Enter the number of pipes: 6
Enter the details of the pipes (start_house end_house diameter):
7 4 98
5 9 72
4 6 10
2 8 22
9 7 17
3 1 66
3
2 8 22
3 1 66
5 6 10
```

Program 13:- Write a program to implement Floyd Warshall Algorithm.

```
#include <iostream>
#include <vector>
using namespace std;
#define INF 99999
void printSolution(vector<vector<int>>& dist, int V) {
   cout << "The following matrix shows the shortest distances between every pair of vertices:\n";
   for (int i = 0; i < V; i++) {
      for (int j = 0; j < V; j++) {
         if (dist[i][j] == INF)
            cout << "INF ";
         else
            cout << dist[i][j] << " ";
      }
      cout << endl;
   }
}
void floydWarshall(vector<vector<int>>& dist, int V) {
   for (int k = 0; k < V; k++) {
      for (int i = 0; i < V; i++) {
         for (int j = 0; j < V; j++) {
            if (dist[i][j] > dist[i][k] + dist[k][j]
               && dist[i][k] != INF && dist[k][j] != INF) {
               dist[i][j] = dist[i][k] + dist[k][j];
            }
         }
      }
   }
   printSolution(dist, V);
}

int main() {
   int V;
   cout << "Enter the number of vertices: ";
   cin >> V;

   vector<vector<int>> graph(V, vector<int>(V));

   cout << "Enter the adjacency matrix of the graph (use " << INF << " for no direct connection):\n";
   for (int i = 0; i < V; i++) {
      for (int j = 0; j < V; j++) {
         cin >> graph[i][j];
      }
   }

   floydWarshall(graph, V);

   return 0;
}
```

Output:-

```
Enter the number of vertices: 4
Enter the adjacency matrix of the graph (use 99999 for no direct
    connection):
0 5 99999 10
99999 0 5 99999
99999 99999 0 1
99999 99999 99999 0
The following matrix shows the shortest distances between every pair of
    vertices:
0 5 10 10
INF 0 5 6
INF INF 0 1
INF INF INF 0
```